

ITERATORS, GENERATORS, AND OBJECTS Meta

COMPUTER SCIENCE MENTORS 61A

March 10, 2026 – March 13, 2026

Example Timeline

- Teaching tips:

Each problem's "Suggested Time" is not assigned with the 1 hour time for sections in mind. It is more of a note of how long each individual problem may take. That is, if you added all the "Suggested Times" in the coding section, it could very well be greater than 1 hour (especially in more complicated worksheets in the future).

However, the time in brackets will add up to 50 minutes, and is intended to be a suggested outline for your section.

Also, the worksheet feedback form is linked at the bottom of this worksheet!

- Intros, Ice Breakers, & Logistics [2 min]

Start off with an icebreaker of your choice!

- Mini-lecture: Iterators [5 min]
- Q1: Iterators WWPD [5 min]
- Q2: Map WWPD [3 min]
- Mini-Lecture: Generators [5 min]
- Q3: Generator WWPD [5 min]
- Q4: Accumulator [5 min]
- Q5: Filter Generator [7 min]
- Mini-Lecture: Objects [5 min]
- Q6: Car Object [5 min]
- Optional: Iterator Puzzle [If Extra Time]

1. What Would Python Display? If you think nothing will be displayed, write N/A.

```
>>> a = [1, 2, 3]
>>> x = iter(a)
```

N/A

```
>>> next(x)
```

1

```
>>> next(x)
```

2

```
>>> y = iter(a)
>>> next(y)
```

1

```
>>> next(x)
```

3

```
>>> next(x)
```

StopIteration Error

```
>>> z = iter(y)
>>> next(z)
```

2

```
>>> [next(y), next(y), next(z)]
```

[2, 3, 3]

Suggested Time: 5 min

2. What Would Python Display? If you think nothing will be displayed, write N/A.

```
>>> x = [1, 2, 3]
>>> y = map(lambda x : x + 10, x)
```

N/A

```
>>> next(y)
```

11

```
>>> [z for z in y]
```

[12, 13]

```
>>> next(y)
```

StopIteration

Teaching Tips

- This question is a quick introduction to the map function.

Suggested Time: 3 min

2 Generators

1. Given the following code block, what is output by the lines that follow?

```
def foo():
    a = 0
    if a == 0:
        print("Hello")
        yield a
        print("World")
```

```
>>> foo()
```

<generator object>

```
>>> foo_gen = foo()
>>> next(foo_gen)
```

Hello
0

```
>>> next(foo_gen)
```

```
World  
StopIteration
```

```
>>> for i in foo():  
...     print(i)
```

```
Hello  
0  
World
```

Teaching Tips

- Emphasize heavily the fact that when generators are called, they return a generator object. They do NOT start executing their function body until after `next` is called! (So what does that first line return? A generator object!)
- Remind students that generator objects are independent from one another; if you create a new one from calling the same function again, it starts from the beginning again. Each generator on its own, however, remembers where it stopped after the previous `next` call, so that it can resume the next time you call `next`.
- What happens when there are no more `yield` statements, like in the second call on `foo_gen`? The generator has reached the end of all possible values to iterate over, and so it returns a `StopIteration` error.
- If you stick a generator object inside a `for` loop (or a list, for that matter), it will go all the way through from start to finish, outputting each `yield` value after another.
 - Careful, however: ‘start’ doesn’t necessarily mean the very first lines of the function or the first `yield` call; if you feed in a generator on which you’ve already called `next`, its “start” will be where it last left off.

Suggested Time: 5 min

2. Write a generator function that takes in an iterator `it` and yields the running total of the elements produced by it.

```
def accumulate(it):  
    '''  
    >>> def all_ints():  
    ...     i = 0  
    ...     while True:  
    ...         yield i  
    ...         i += 1  
    >>> a = accumulate(all_ints())  
    >>> [next(a) for x in range(6)]  
    [0, 1, 3, 6, 10, 15]  
    '''
```

```
def accumulate(it):  
    sum = 0  
    while True:  
        sum += next(it)  
        yield sum
```

This problem is meant to be a relatively simple introduction to the manipulation of iterators, generator function syntax, and how to deal with infinite generators.

The doc test for this question is deliberately a bit complicated. It actually serves as a bit of a hint for how we might construct an infinite generator if one is provided to us (using an infinite loop). If students are concerned about this possibly erroring, remind them that generators only execute the body of the function when we tell them to, so this will not actually cause an infinite loop that hogs all of our computer's resources, causing a crash.

Students have a tendency to want to do everything in generators with lists, which requires the computation of all elements of the generator at the same time; we need to encourage them to do "lazy" iteration, which means that each element is only computed when it's needed. Lazy iteration is a core concept of the topic; not only is it more efficient to do things this way, it's literally the only way to handle infinite iterators (whose elements cannot all be computed at the same time). Whenever I do iteration problems with my students, I ask them to consider what would happen if we provided them an infinite generator; if their implementation breaks, then it's not correct.

Suggested Time: 5 min

3. Define `filter_gen`, a generator that takes in iterable `s` and one-argument function `f` and yields every value from `s` for which `f` returns a truthy value.

```
def filter_gen(s, f):
    """
    >>> list(filter_gen([1, 2, 3, 4, 5],
                        lambda x: x % 2 == 0))
    [2, 4]
    >>> list(filter_gen([1, 2, 3, 4, 5], lambda x: x < 3))
    [1, 2]
    """

    for x in s:
        if f(x):
            yield x
```

Teaching Tips

- It may look complicated, but the structure of the function is actually very similar to other functions that have for loops and if statements! Think about how you can iterate through each element in `s`, and then think about how you can apply the filter to the elements.
- Since if conditionals rely on boolean values, what can you put in your if statement to filter out the list elements?
- Now the only thing left to do, after you've ensured that you've designed a way to get only the elements you want, is to yield them! Yielding automatically turns your function into a generator (make sure students know this!).

Suggested Time: 7 min

3 Objects

1. Given the following code, what will Python output for the following prompts? If you think nothing will be outputted, write N/A.

```
class Car:
    wheels = 4
    def __init__(self, gas):
        self.gas = gas

    def drive(self):
        if self.gas <= 0:
            print("Out of gas")
        else:
            self.gas -= 1

    def fill(self, gas):
        self.gas += gas
```

```
>>> c1 = Car(2)
>>> c1.gas
```

2

```
>>> c1.drive()
```

N/A

```
>>> c1.gas
```

1

```
>>> c1.drive()
>>> c1.drive()
```

Out of gas

```
>>> c1.fill(10)
>>> c1.drive()
>>> c1.gas
```

9

```
>>> c1.wheels
>>> Car.wheels
```

44

Suggested Time: 5 min

4 Optional

1. Find an input to the year function that prints the following output: 2 0 2 6

```
def year(a):
    x = iter(a)
    y = iter(a)
    z = iter(x)
    for i in range(next(x)):
        y = iter(a)
        next(y)
    print(next(x))
    print(next(z))
    print(next(y))
    print(next(z))
```

```
year([_, 2, 0, 6, _])
```

There can be **any** number **in** the first blank, **and** there can be **any** number of values after the 6.

Teaching Tips

- The first step to this problem is drawing out the initial states of the x and y iterators as pointers under the element at index 0 of an unknown list
- Recognize that `iter(x)` returns the iterator x itself, so z is x.
- Note that the for loop header moves the x iterator's position up by one. The body of the for loop assigns y to a new `iter(a)` so no matter how many times the for loop runs, the for loop will always end with the y iterator pointing to the item at the first index of a.
- We want the function to print 2, 0, 2, and 6 in that order. We know that the x iterator is pointing to the element at index 1 of the list, so the element at index 1 must be a 2. Calling `print(next(x))` should print 2 and move the x iterator to point to the element at index 2.
- We know that the z iterator is the same as the x iterator, so since it is pointing to the element at index 2, the element at index 2 must be 0. Calling `print(next(z))` should print 2 and also moves the iterator up to point to the item at index 3 in the list.
- We know from the end of the for loop that the y iterator points to the item at index 1, which we want to be 2. Fortunately, we already know that this item is 2 from when we figured out what `print(next(x))` was.
- Lastly, we look at the z iterator (the same as the x iterator) and see its position is under the item at index 3, so we know that the item at index 3 should be 6.

Suggested Time: If Extra Time